# CN-Lec04-Operators

## Operators

### 🎯 Session Objectives

- 🔧 Understand what operators are and why they are used
- 🧮 Explore different types of operators in Python
- 🔢 Learn about operator precedence and order of execution
- ⚠️ Understand constraints in programming

```python
num1 = int(input("Enter the first value: "))
num2 = int(input("Enter the first value: "))
result = num1 + num2
print(f"The final Result will be {result}.")
```

```
Enter the first value:  21
Enter the first value:  56
The final Result will be 77.
```

```python
num1 = int(input("Enter the first value: "))
num2 = int(input("Enter the first value: "))
result = num1 * num2
print(f"The final Result will be {result}.")
```

```
Enter the first value:  11
Enter the first value:  10
The final Result will be 110.
```

```python
print(num1) # 11
print(num2) # 10
print(result) # 110
```

```
11
10
110
```

```python
fav_car = input("Enter your Favorite Car: ")
print(f"My Favorite car is {fav_car}.")
```

```
Enter your Favorite Car:  Hummer
My Favorite car is Hummer.
```

```python
# Identation:
z = 15
if z > 10: # True
    print("z is greater than 10.")
print("Outside the if Statement.")
```

```
z is greater than 10.
Outside the if Statement.
```

```python
# Identation:
z = 15
if z > 20: # False
    print("z is greater than 10.") # Skipped
print("Outside the if Statement.")
```

```
Outside the if Statement.
```

# Variables : What & Why?

**Variables acts as an container for storing data, They help with:**

- Storing Data.
- Manipulating Values
- Reusablitiy
- Improving the Readibility:
  - total_sum >> t_s
  - compound_interest >> compoundInterest >> c_i
  - age
  - finding_prod_sum >> finding_the_product_and_sum >> p_s

```
# Declaring and assigning variables:
val = 99
user_name = "UltimateForce"
print(val)
print(user_name)
```

```
99
UltimateForce
```

```
a,b,c = 10,20,30
print(a) # 10
print(b) # 20
print(c) # 30
```

```
10
20
30
```

```
# * arbitary argument
a,b,*c = 10,20,30,40,50,60,70
print(a) # 10
print(type(a)) # 'int'
print(b) # 20
print(type(b)) # 'int'
print(c) # [30,40,50,60,70]
print(type(c)) # 'List'
```

```
10
<class 'int'>
20
<class 'int'>
[30, 40, 50, 60, 70]
<class 'list'>
```

```
# * arbitary argument
a,*b,c = 10,20,30,40,50,60,70
print(a) # 10
print(type(a)) # 'int'
print(b) # [20,30,40,50,60]
print(type(b)) # 'List'
print(c) # 70
print(type(c)) # 'int'
```

```
10
<class 'int'>
[20, 30, 40, 50, 60]
<class 'list'>
70
<class 'int'>
```

```python
# * arbitary argument
a,b,*c = 10,20,30
print(a) # 10
print(type(a)) # 'int'
print(b) # 20
print(type(b)) # 'int'
print(c) # [30]
print(type(c)) # 'list'
```

```
10
<class 'int'>
20
<class 'int'>
[30]
<class 'list'>
```

```python
# * arbitary argument
a,b,*c = 10,20,30,40,50,60,70,'a','c','e'
print(a) # 10
print(type(a)) # 'int'
print(b) # 20
print(type(b)) # 'int'
print(c) # [30,40,50,60,70,'a','c','e'] # Mixed Data Types
print(type(c)) # 'list'
```

```
10
<class 'int'>
20
<class 'int'>
[30, 40, 50, 60, 70, 'a', 'c', 'e']
<class 'list'>
```

```python
# * arbitary argument
a,*b,*c = 10,20,30,40,50,60,70,'a','c','e'
print(a) # 10
print(type(a)) # 'int'
print(b) # 20
print(type(b)) # 'int'
print(c) # [30]
print(type(c)) # 'list'
# SyntaxError: multiple starred expressions in assignment
```

```python
p=q=r=s="Python Programming"
print(p)
print(q)
print(r)
print(s)
```

```
Python Programming
Python Programming
Python Programming
Python Programming
```

# Rules for naming a "Variables"

- Can include letters , digits and underscore.
- Must Start the Variable with either a letter or underscore.
- Case - Sensitive
  - val != Val
  - num != nuM
  - digit != Digit
- Can't use the Python Keyword ....
  - if,elif,def,tuple,list,type,while,class,len,dict,True,False,input
- Can't have spaces or special characters except (_ underscore)
- Variable name can't start with numbers.

```
my_first_variables = "Programming"
print(my_first_variables)
```

```
Programming
```

```
_var = "Coding"
print(_var)
```

```
Coding
```

```
# type() Conversion
x = 10.0
print(x) # 10.0
print(type(x)) # 'float'
x = int(x) # 'int'
print(x) # 10
print(type(x)) # 'int'
```

```
10.0
<class 'float'>
10
<class 'int'>
```

```
x = '55'
print(x) # '55'
print(type(x)) # 'str'
x = int(x) # 'int'
print(x) # 55
print(type(x)) # 'int'
```

```
55
<class 'str'>
55
<class 'int'>
```

```
_bool = True
print(_bool)
print(type(_bool)) # boolean
_bool = int(_bool) # 1
print(_bool) # 1
print(type(_bool)) # 'int'
```

```
True
<class 'bool'>
1
<class 'int'>
```

```python
_bool = False
print(_bool)
print(type(_bool)) # boolean
_bool = int(_bool) # 0
print(_bool) # 0
print(type(_bool)) # 'int'
```

```
False
<class 'bool'>
0
<class 'int'>
```

## What are Operators?

**Operators are tools in programming used to perform actions like arithmetic, comparisons, assignments, logical evaluations, etc.**

### Types of Operators in Python:

1. Arithmetic Operators
2. Comparison Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Membership Operators
7. Identity Operators

## Arithmetic Operators :

- '+' Addition
- '-' Subtraction
- '*' Multiplication
- '/' Division floating
- '%' Modulous
- '**' Exponentiation
- '//' Floor Division

```python
x = 11
y = 7
z = 15
print(x+y) # 18
print(x-y) # 4
print(x*y) # 77
print(x/y) # 1.57
print(x%y) # 4
print(z%y) # 1
print(z%x) # 4
print(x**2) # 121
print(x**3) # 1331
print(x//2) # 5
```

```
18
4
77
1.5714285714285714
4
1
4
121
1331
5
```

## Comparison Operators (Return Boolean Result)

- '==' Equal To
- '!=' Not Equal To
- '>' Greater Than
- '>=' Greater Than or Equal To
- '<' Less Than
- '<=' Less Than or Equal To

```python
x = 11
y = 7
z = 15
print(x==y) # False
print(x!=y) # True
print(z!=y) # True
print(z!=15) # False
print(x>y) # True
print(x>=y) # True
print(z<=y) # False
print(z<x) # False
print(x!=2) # True
print(x>=3) # True
```

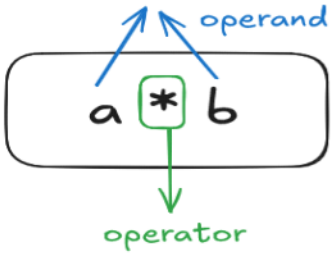
```
False
True
True
False
True
True
False
False
True
True
```

## What are Operators?

Operators are tools in programming used to perform actions like arithmetic, comparisons, assignments, logical evaluations, etc.

## Types of Operators in Python:

1. Arithmetic Operators
2. Comparison Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Membership Operators
7. Identity Operators



# Logical Operators: ('not' > 'and' > 'or') ¶

- 'and' : Returns True, if both the conditions are True else False
- 'or' : Returns False, if both the conditions are False, else True
- 'not' : Returns the Opposite.

```
x = 11
y = 7
z = 15
print((x==y) and (z!=y) or (y>x)) # (F and T or F) # (F or F) # F
print((x!=y) or (z!=y) or (y>x)) # (T or T or F) # (T or F) # T
print((x>=y) and (z==y) and (z>x)) # (T and F or T) # (F and T) # F
print(not(z!=15)) #Not(F) # T
print(not((x<=y) and (z>=y) or (y!=x))) # not(F and T or T) # not(F or T) # not(T) # ~T = F
```

```
False
True
False
True
False
```

```
print(not(x<=y) and (z>=y) or (y!=x)) # not(F) and T or T  # T and T or T # True
```

```
True
```

```
print(not(x<=y) or (z<=y) and (y!=x)) # not(F) or (F) and T
# T or F and T # T or F # True
```

```
True
```

Logical Operators

| cond1 | cond2 | result |
|---|---|---|
| T | T | T |
| T | F | F |
| F | T | F |
| F | F | F |

AND Logic

| cond1 | cond2 | result |
|---|---|---|
| T | T | T |
| T | F | T |
| F | T | T |
| F | F | F |

OR Logic

| cond1 | Result |
|---|---|
| T | F |
| F | T |

NOT Logic

Bitwise Operators

# Bitwise Operators:

- 'AND' = '&' - 'Both bits with value 1 return 1 else 0'
- 'OR' = '|' - 'Any bit with value 1 return 1 else 0'
- 'XOR' = '^' - 'Alternative Bits returns 1 else same bit returns 0'
- 'NOT' = '~' - '2's complement to check the sign and evaluate'
- 'Left Shift' = '<<' - 'Shifts the bit to the left'
- 'Right Shift' = '>>' - 'Shifts the bit to the right'

| bit1 | bit2 | result |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

AND & Bitwise

| bit1 | bit2 | result |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 0 |

OR | Bitwise

| bit1 | bit2 | result |
|---|---|---|
| 1 | 1 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 0 |

XOR ^ Bitwise